

高等數位訊號處理

final project

學號：M11202169

姓名：鍾坤佑

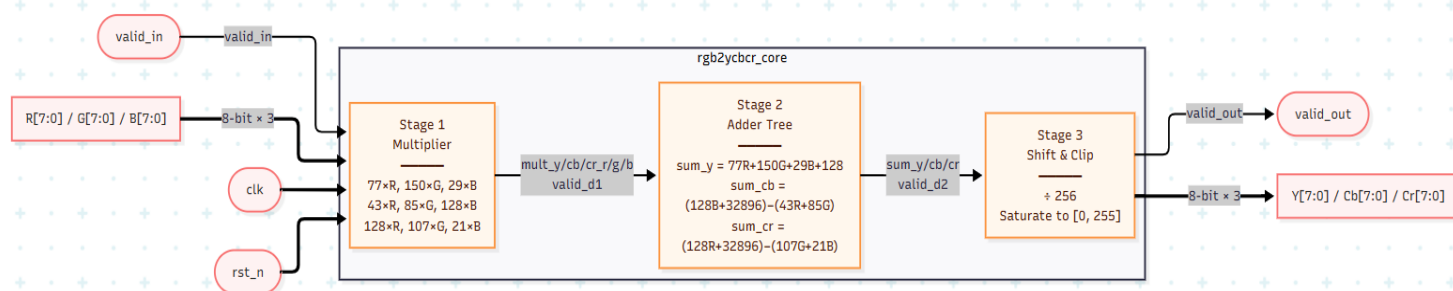
Color Coordinate Transform	3
1. RGB to YCbCr core	3
2. YCbCr 444 to 420 drop	5
3. Top module	6
Python and Verilog Code Simulation	7
1. Image to RGB (python)	7
2. RGB to YCbCr core (verilog).....	8
3. YCbCr 444 to 420 drop (verilog)	12
4. Top module (verilog)	15
5. verify 4:2:0 and compute PSNR (python).....	19
Discussion	22

Color Coordinate Transform

這次的 project 我選擇做 color coordinate transform。在影像處理中，由於高解析度的 RGB 影像會帶來龐大的資料流與運算負擔，所以必須將 RGB 的影像轉換至 YCbCr 空間，轉換過後，由於人類視覺系統對亮度 Y 的敏感度遠高於色度 CbCr，所以我們通常會將搭配 4:2:0 降採樣 (chroma subsampling) 技術，藉此成功捨棄掉 75% 的色度資料，做到影像的預處理，這也是 JPEG 影像壓縮技術的前幾個步驟。

在程式模擬時，我先用 python 將原始影像每個 pixel 24 個位元的 RGB 資料輸出成文字檔，每一行代表一個 pixel，所以一個 1350x1080 像素的影像就會產生出 1458000 行的文字檔，接著將這份文字檔當作 verilog 硬體模擬的測試資料，把 RGB 的輸入轉換為 YCbCr，再根據 4:2:0 的 chroma subsampling 把 2D 的 pixel 四個為一組，只保留左上角 pixel 的 CbCr，用這種方式就可以做到在維持視覺無損的條件下大幅降低記憶體頻寬。

1. RGB to YCbCr core



首先介紹這個 RGB 轉 YCbCr 的核心運算模組。已知標準的轉換公式充滿了像 0.299、0.587 這樣的浮點數，但在硬體裡面，實作浮點運算單元 (FPU) 是非常耗費晶片面積的，那為了解決這個問題，我採用了定點數量化的技巧，就是將整個浮點數矩陣同乘 256 倍 (或是乘以 1024 倍也可以)，這樣就可以把所有的權重都轉換成整數，例如，0.299 乘上 256 後大約是 76.5，我將其四捨五入取 77，

這樣一來，原本昂貴的浮點數乘法，就被降級成了面積極小的 8-bit 整數乘法器。

Stage 1 pipeline：乘法陣列

接著，為了確保資料能夠在高速時脈下運作，我將演算法拆解成 3-Stage 的 pipeline 架構，第一級專心處理乘法，當 valid 訊號進來後，硬體會將 R、G、B 分別乘上我剛剛算好的整數權重，且為了防止乘法運算溢位，我將原本 8-bit 的資料，擴展到了 16-bit 來暫存結果。

Stage 2 pipeline：加法樹

在這裡有兩個非常關鍵的硬體巧思：

- 四捨五入的零成本補償：我們知道把數值放大 256 倍後，最後一定要除以 256。在整數除法中，直接捨去小數點會造成嚴重的色彩變暗，所以我在 Y 通道的加法樹中，先加上了 128 這個常數 (256 的一半)，這等同於在除以 256 之前先加上 0.5，以此實現硬體的四捨五入。而 Cb 與 Cr 通道因為本來就需要加上 128 的色彩偏移量，所以我將公式裡的 128 乘上 256，再加上 128 的補償，算出了 32896 這個常數也要加到 Cb、Cr 裡，最後再去除以 256。
- 防止無號數下溢：Cb 與 Cr 的公式中包含減法，如果直接照公式減去的話，運算中間可能會出現負數，這在無號數中會導致嚴重的錯誤，所以我先加總所有正數與 offset，最後再去扣掉負數的部分，這樣確保了線路上的每一根 wire 永遠都是絕對正值。

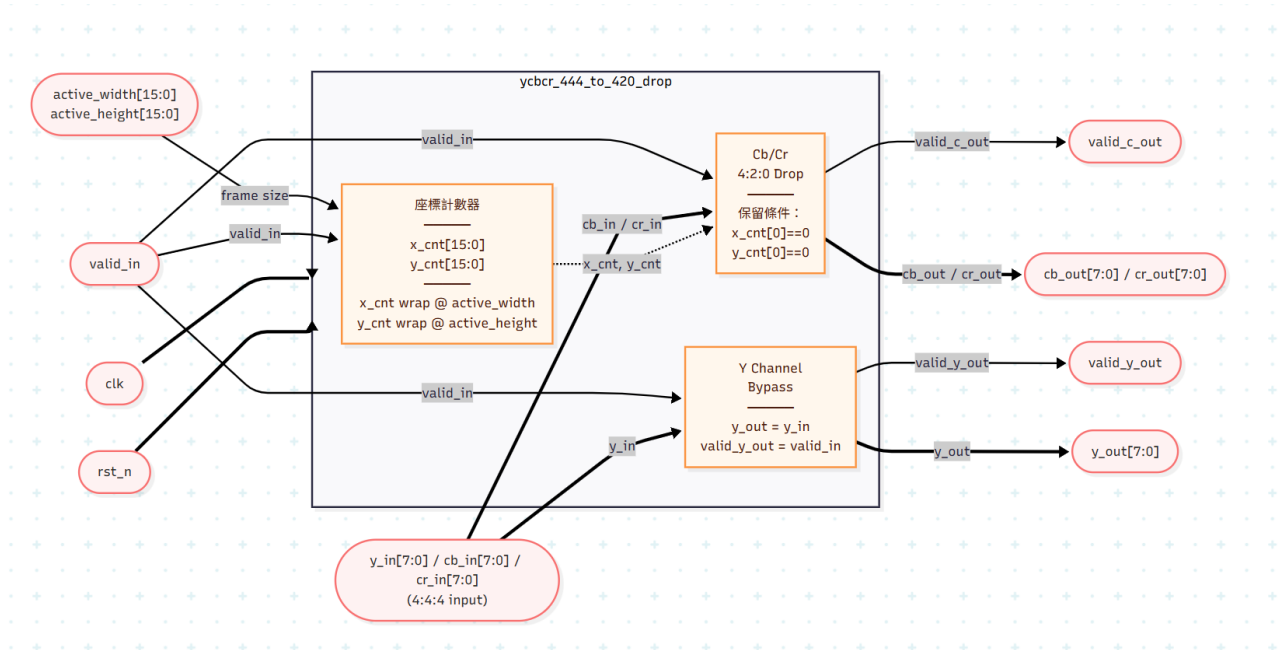
Stage 3 pipeline：移位與零成本截斷

最後一級 pipeline，要把放大 256 倍的數值還原回來。

- 使用右移取代除法：在 verilog 裡，除法不可以直接寫 /，因為這會合成出超大的電路，所以我直接提取運算結果的 [15:8] 位元，這等同於完成了除以 256 的動作，且在硬體上只是單純的重新接線，完全不需要消耗任何邏輯閘。
- 飽和截斷：如果算出來的值超過 255 怎麼辦？在這裡，我沒有使用佔用面積的比較器 (ex: if (sum > 255))，因為第二級 pipeline 使用的是 18-bit 的暫存器，所以只需要判斷第 16 與 17 位元是否不為零 (也就是大於等於 65536)，也就是只要一個簡單的 or 邏輯 |sum_y[17:16]，只要這個條件成

立，我就直接透過 mux 輸出 8'hFF (等於 255)，確保影像亮點不會因為溢位而變成黑點。

2. YCbCr 444 to 420 drop



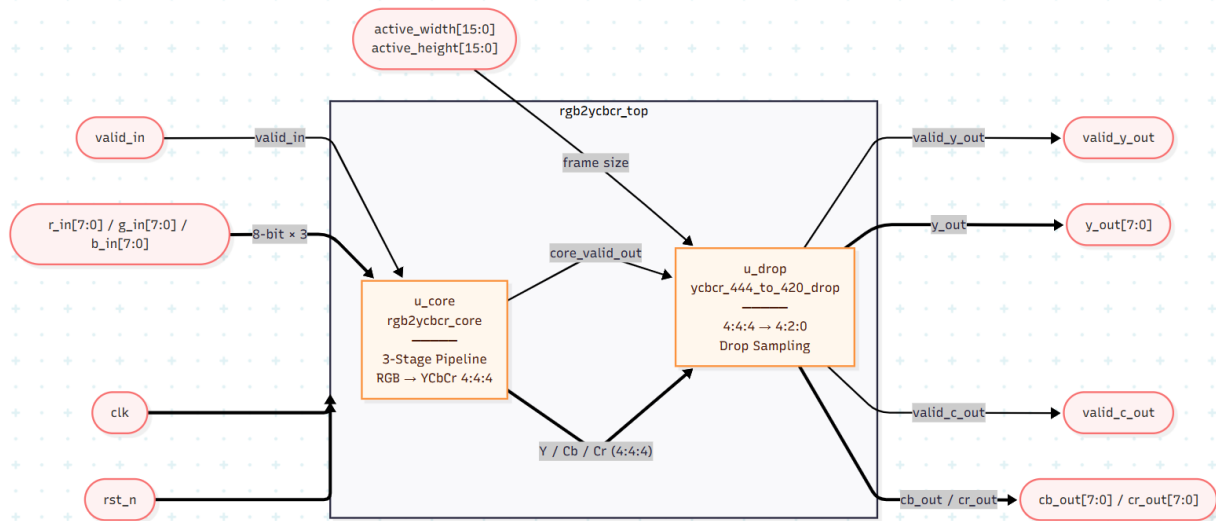
接下來是 4:2:0 降採樣模組，由於硬體不能輕易使用雙層 for 迴圈來遍歷整張圖片，資料是一直像流水一樣灌進來的，所以為了讓硬體知道現在這筆資料到底在圖片的哪個位置，我就設計了 `x_cnt` 與 `y_cnt` 的空間座標計數器，他可以透過接收 CPU 動態設定的 `active_width` 與 `active_height`，計數器能精準判斷何時該換行、何時該回到原點。

另外，這個模組最大的挑戰是 Y 通道的亮度資料是連續無損輸出的，但 CbCr 色度資料卻要刪除 3/4，這在硬體上，其實就如同處理非對稱週期的時脈除頻器邏輯一樣，需要精準掌控資料的有效週期。因此，我將原本單一的輸出的 `valid` 訊號，拆分成了獨立的 `valid_y_out` 與 `valid_c_out`，Y 通道直接通過，而 CbCr 通道則只在特定的時序邊緣才會拉高 `valid`，讓下面知道何時該抓取資料。

最後，在 4:2:0 的 drop 策略裡，我們只需要保留 2x2 區塊中左上角的色度像素。在數位邏輯中，要判斷座標是否為偶數，如果寫成 `% 2 == 0` 也會合成出

極度浪費面積的除法器，所以我的做法是直接提取計數器的最低位元（LSB），只要 $x_cnt[0]$ 與 $y_cnt[0]$ 同時為 0，就代表這是偶數行與偶數列的交界，也就是左上角，透過這個簡單的判斷就不需要消耗任何 SRAM（line buffer），用極低的硬體成本成功將色度資料的傳輸頻寬降了 75%，而在無效的週期，就可以將輸出清零（8'd0）了。

3. Top module



最後，是這個專案的 top module，作用就是提供一個乾淨的介面：輸入端只要餵標準的 RGB 訊號進來，輸出端就會自動吐出分離好的 Y 亮度通道與 CbCr 色度通道。在內部實體連線的部分，我宣告了 `core_valid_out` 等訊號，將前面運算核心的輸出，對接給後面的降採樣模組，一筆 RGB 資料進來後，會在第一個模組經歷 3 個 clock cycles 的運算延遲，接著透過內部 wire 傳遞，在第二個模組經歷 1 個 clock cycle 的控制邏輯延遲。

另外，我也在頂層宣告了 `MAX_WIDTH` 和 `MAX_HEIGHT` 這兩個 parameter，並在例化子模組時將它們一路往下傳遞，這也給了這份專案彈性的變化。可以在需要不同解析度時，最小幅度的修改底層的 verilog code，只要在呼叫這個 top module 時修改參數即可。

Python and Verilog Code Simulation

1. Image to RGB (python)

```
1 from PIL import Image
2 import numpy as np
3 import matplotlib.pyplot as plt
4
5 # =====
6 # 讀取圖片
7 # =====
8 img_path = 'road.jpg' # 替換圖片檔名
9 img = Image.open(img_path).convert('RGB')
10
11 img_array = np.array(img)
12
13 print(f"圖片解析度 (Height, Width, Channels): {img_array.shape}")
14 print(f"資料型態: {img_array.dtype} (通常是 uint8, 範圍 0~255)")
15
16 first_pixel = img_array[0, 0]
17 print(f"左上角第一顆像素的 RGB 值: R={first_pixel[0]}, G={first_pixel[1]}, B={first_pixel[2]}")
18
19 # =====
20 # 準備 verilog 測資
21 # =====
22 # 把 3D 陣列攤平成 1D, 並轉成 Hex 寫入 txt 檔 (ex: FF8800)
23 output_filename = 'input_rgb_hex.txt'
24
25 with open(output_filename, 'w') as f:
26     flat_pixels = img_array.reshape(-1, 3)
27
28     for pixel in flat_pixels:
29         r, g, b = pixel
30         hex_str = f"{r:02X}{g:02X}{b:02X}\n"
31         f.write(hex_str)
32
33 print(f"\n成功! 已將 {len(flat_pixels)} 個像素的 RGB 資料寫入 {output_filename}")
34 print("這份 txt 檔案未來可以直接用 Verilog 的 $readmemh 或 $fscanf 讀取!")
35
```

原始圖片：



成功轉換後的 RGB .txt 檔：



2. RGB to YCbCr core (verilog)

Verilog :

```
23 module rgb2ycbcr_core(
24     input wire      clk,
25     input wire      rst_n,
26
27     input wire      valid_in,
28     input wire [7:0] r_in,
29     input wire [7:0] g_in,
30     input wire [7:0] b_in,
31
32     output reg      valid_out,
33     output reg [7:0] y_out,
34     output reg [7:0] cb_out,
35     output reg [7:0] cr_out
36 );
37
38 // =====
39 // 定義 (放大 256 倍)
40 // =====
41 // Y = 0.299*R + 0.587*G + 0.114*B -> 77*R + 150*G + 29*B
42 // Cb = -0.169*R - 0.331*G + 0.500*B -> -43*R - 85*G + 128*B + 32768
43 // Cr = 0.500*R - 0.419*G - 0.081*B -> 128*R - 107*G - 21*B + 32768
44
45 // =====
46 // Pipeline Stage 1: Multiplier
47 // =====
48 reg      valid_d1;
49 reg [15:0] mult_y_r, mult_y_g, mult_y_b;
50 reg [15:0] mult_cb_r, mult_cb_g, mult_cb_b;
51 reg [15:0] mult_cr_r, mult_cr_g, mult_cr_b;
52
53 always @(posedge clk or negedge rst_n) begin
54     if (!rst_n) begin
55         valid_d1 <= 1'b0;
56         mult_y_r <= 16'd0; mult_y_g <= 16'd0; mult_y_b <= 16'd0;
57         mult_cb_r <= 16'd0; mult_cb_g <= 16'd0; mult_cb_b <= 16'd0;
58         mult_cr_r <= 16'd0; mult_cr_g <= 16'd0; mult_cr_b <= 16'd0;
59     end else begin
60         valid_d1 <= valid_in;
61
62         if (valid_in) begin
63             // Y 乘積
64             mult_y_r <= 8'd77 * r_in;
65             mult_y_g <= 8'd150 * g_in;
66             mult_y_b <= 8'd29 * b_in;
67
68             // Cb 乘積
69             mult_cb_r <= 8'd43 * r_in;
70             mult_cb_g <= 8'd85 * g_in;
71             mult_cb_b <= 8'd128 * b_in;
72
73             // Cr 乘積
74             mult_cr_r <= 8'd128 * r_in;
75             mult_cr_g <= 8'd107 * g_in;
76             mult_cr_b <= 8'd21 * b_in;
77         end
78     end
79 end
```



```

81 // =====
82 // Pipeline Stage 2: Adder Tree
83 // =====
84 reg valid_d2;
85 // 使用 18-bit 確保加法不溢位
86 reg [17:0] sum_y;
87 reg [17:0] sum_cb;
88 reg [17:0] sum_cr;
89
90 // offset 32768 + 四捨五入補償 128 = 32896
91 wire [17:0] ROUNDING_OFFSET = 18'd128;
92 wire [17:0] CHROMA_OFFSET = 18'd32896;
93
94 always @(posedge clk or negedge rst_n) begin
95     if (!rst_n) begin
96         valid_d2 <= 1'b0;
97         sum_y <= 18'd0;
98         sum_cb <= 18'd0;
99         sum_cr <= 18'd0;
100     end else begin
101         valid_d2 <= valid_d1;
102
103         if (valid_d1) begin
104             // Y = 77R + 150G + 29B + 128
105             sum_y <= mult_y_r + mult_y_g + mult_y_b + ROUNDING_OFFSET;
106
107             // Cb = 128B + 32896 - (43R + 85G)
108             sum_cb <= (mult_cb_b + CHROMA_OFFSET) - (mult_cb_r + mult_cb_g);
109
110             // Cr = 128R + 32896 - (107G + 21B)
111             sum_cr <= (mult_cr_r + CHROMA_OFFSET) - (mult_cr_g + mult_cr_b);
112         end
113     end
114 end

116 // =====
117 // Pipeline Stage 3: Shift & Clip 如果大於等於 65536 (bit[17:16] != 0) ,
118 // 強制設為 255 , 否則取 bit[15:8] (相當於除以 256)
119 // =====
120 always @(posedge clk or negedge rst_n) begin
121     if (!rst_n) begin
122         valid_out <= 1'b0;
123         y_out <= 8'd0;
124         cb_out <= 8'd0;
125         cr_out <= 8'd0;
126     end else begin
127         valid_out <= valid_d2;
128
129         if (valid_d2) begin
130             if (|sum_y[17:16]) y_out <= 8'hFF;
131             else y_out <= sum_y[15:8];
132
133             if (|sum_cb[17:16]) cb_out <= 8'hFF;
134             else cb_out <= sum_cb[15:8];
135
136             if (|sum_cr[17:16]) cr_out <= 8'hFF;
137             else cr_out <= sum_cr[15:8];
138         end
139     end
140 end
141 endmodule

```

Testbench :

```
23 module tb_rgb2ycbcr_core;
24
25     parameter CLK_PERIOD = 10;
26
27     reg        clk;
28     reg        rst_n;
29     reg        valid_in;
30     reg [7:0]  r_in;
31     reg [7:0]  g_in;
32     reg [7:0]  b_in;
33
34     wire        valid_out;
35     wire [7:0]  y_out;
36     wire [7:0]  cb_out;
37     wire [7:0]  cr_out;
38
39     integer fd_in;
40     integer fd_out;
41     integer scan_res;
42
43     // 暫存讀取到的一行 24-bit RGB 十六進位資料
44     reg [23:0] rgb_data;
45
46     rgb2ycbcr_core u_core (
47         .clk        (clk),
48         .rst_n       (rst_n),
49         .valid_in    (valid_in),
50         .r_in        (r_in),
51         .g_in        (g_in),
52         .b_in        (b_in),
53         .valid_out   (valid_out),
54         .y_out       (y_out),
55         .cb_out      (cb_out),
56         .cr_out      (cr_out)
57     );
58
59     initial begin
60         clk = 0;
61         forever #(CLK_PERIOD / 2) clk = ~clk;
62     end
63
64     // =====
65     // 監聽結果並寫入輸出檔案
66     // =====
67     initial begin
68         fd_out = $fopen("C:/Users/david/Desktop/Color_Coordinate_Transform/output_ycbcr_hex.txt", "w");
69         if (fd_out == 0) begin
70             $display("Error: Failed to create or open output_ycbcr_hex.txt");
71             $finish;
72         end
73
74         forever begin
75             @(posedge clk);
76             if (valid_out) begin
77                 // 以十六進位格式寫入，例如 "FF8080"
78                 $fwrite(fd_out, "%02X%02X%02X\n", y_out, cb_out, cr_out);
79             end
80         end
81     end
82
83     // =====
84     // 讀取輸入檔並模擬
85     // =====
86     initial begin
87         rst_n = 0;
88         valid_in = 0;
89         r_in = 0;
90         g_in = 0;
91         b_in = 0;
92
93         // 開啟輸入檔案
94         fd_in = $fopen("C:/Users/david/Desktop/Color_Coordinate_Transform/road.txt", "r");
95         if (fd_in == 0) begin
96             $display("Error: input_rgb_hex.txt not found. Please check the file path.");
97             $finish;
98         end
99     end
```

```

100      #(CLK_PERIOD * 2);
101      rst_n = 1;
102      #(CLK_PERIOD * 2);
103
104      while (!$feof(fd_in)) begin
105          @(negedge clk);
106
107          scan_res = $fscanf(fd_in, "%h\n", rgb_data);
108
109          if (scan_res == 1) begin
110              valid_in = 1;
111              r_in = rgb_data[23:16];
112              g_in = rgb_data[15:8];
113              b_in = rgb_data[7:0];
114          end else begin
115              valid_in = 0;
116          end
117      end
118
119      @(negedge clk);
120      valid_in = 0;
121
122      #(CLK_PERIOD * 5);
123
124      $fclose(fd_in);
125      $fclose(fd_out);
126      $display("=====");
127      $display("Hardware simulation completed!");
128      $display("Please return to Python to check output_ycbcr_hex.txt");
129      $display("=====");
130      $finish;
131  end
132 endmodule

```

原始 RGB .txt 檔：



```

road.txt
檔案 編輯 檢視
567297
4C688D
4C688D
516D92
4E6A8F
506C91
536F94
506C91
536F94
536F94
536F94
536F94
536F94
536F94
536F94
536F94

```

轉換後的 YCbCr .txt 檔：



```

ycbcr_road.txt
檔案 編輯 檢視
6e976f
64976f
64976f
69976f
66976f
68976f
6b976f
68976f
6b976f
6b976f
6b976f
6b976f
6b976f
6b976f
6b976f
6b976f

```

手算驗證第一行的 Y：

$$110 (6e) = [256 * 0.299 * 86 (56) + 256 * 0.587 * 114 (72) + 256 * 0.114 * 151 (97) + 128] / 256 = 110$$

驗證成功。

3. YCbCr 444 to 420 drop (verilog)

Verilog :

```
23 module ycbcr_444_to_420_drop #(
24     parameter MAX_WIDTH  = 1920,
25     parameter MAX_HEIGHT = 1080
26 ) (
27     input wire      clk,
28     input wire      rst_n,
29
30     input wire [15:0] active_width,
31     input wire [15:0] active_height,
32
33     // Input 4:4:4
34     input wire      valid_in,
35     input wire [7:0] y_in,
36     input wire [7:0] cb_in,
37     input wire [7:0] cr_in,
38
39     // Output 4:2:0
40     output reg      valid_y_out,
41     output reg [7:0] y_out,
42     output reg      valid_c_out,
43     output reg [7:0] cb_out,
44     output reg [7:0] cr_out
45 );
46
47 // =====
48 // 座標計數器 (X, Y)
49 // =====
50 reg [15:0] x_cnt;
51 reg [15:0] y_cnt;
52
53 always @(posedge clk or negedge rst_n) begin
54     if (!rst_n) begin
55         x_cnt <= 16'd0;
56         y_cnt <= 16'd0;
57     end else if (valid_in) begin
58         // 水平計數器：數到當前圖片寬度邊界
59         if (x_cnt == active_width - 16'd1) begin
60             x_cnt <= 16'd0;
61
62             // 垂直計數器：換行時 +1
63             if (y_cnt == active_height - 16'd1) begin
64                 y_cnt <= 16'd0; // 一整張圖片掃描完畢，回到原點
65             end else begin
66                 y_cnt <= y_cnt + 16'd1;
67             end
68         end else begin
69             x_cnt <= x_cnt + 16'd1;
70         end
71     end
72 end
```

```

74 // =====
75 // 資料輸出與抽樣邏輯
76 // =====
77 always @(posedge clk or negedge rst_n) begin
78     if (!rst_n) begin
79         valid_y_out <= 1'b0;
80         y_out <= 8'd0;
81         valid_c_out <= 1'b0;
82         cb_out <= 8'd0;
83         cr_out <= 8'd0;
84     end else begin
85         // -----
86         // Y : Bypass
87         // -----
88         valid_y_out <= valid_in;
89         y_out <= y_in;
90
91         // -----
92         // Cb/Cr : 4:2:0 Drop 若 X 和 Y 都是偶數就代表要保留
93         // -----
94         if (valid_in && (x_cnt[0] == 1'b0) && (y_cnt[0] == 1'b0)) begin
95             valid_c_out <= 1'b1;
96             cb_out <= cb_in;
97             cr_out <= cr_in;
98         end else begin
99             valid_c_out <= 1'b0;
100             cb_out <= 8'd0;
101             cr_out <= 8'd0;
102         end
103     end
104 end
105 endmodule

```

Testbench :

```

23 module tb_ybcr_444_to_420_drop;
24
25     parameter CLK_PERIOD = 10;
26
27     parameter IMG_WIDTH = 1080;
28     parameter IMG_HEIGHT = 1350;
29
30     reg clk;
31     reg rst_n;
32     reg [15:0] active_width;
33     reg [15:0] active_height;
34
35     reg valid_in;
36     reg [7:0] y_in;
37     reg [7:0] cb_in;
38     reg [7:0] cr_in;
39
40     wire valid_y_out;
41     wire [7:0] y_out;
42     wire valid_c_out;
43     wire [7:0] cb_out;
44     wire [7:0] cr_out;
45
46     integer fd_in;
47     integer fd_y_out;
48     integer fd_c_out;
49     integer scan_res;
50     reg [23:0] ybcr_data;

```

```

52 ycbcr_444_to_420_drop #(
53     .MAX_WIDTH(1920),
54     .MAX_HEIGHT(1080)
55 ) u_drop (
56     .clk          (clk),
57     .rst_n        (rst_n),
58     .active_width  (active_width),
59     .active_height (active_height),
60     .valid_in      (valid_in),
61     .y_in          (y_in),
62     .cb_in         (cb_in),
63     .cr_in         (cr_in),
64     .valid_y_out   (valid_y_out),
65     .y_out         (y_out),
66     .valid_c_out   (valid_c_out),
67     .cb_out        (cb_out),
68     .cr_out        (cr_out)
69 );
70
71 initial begin
72     clk = 0;
73     forever #(CLK_PERIOD / 2) clk = ~clk;
74 end
75
76 // =====
77 // 監聽結果並寫入輸出檔案
78 // =====
79 initial begin
80     fd_y_out = $fopen("C:/Users/david/Desktop/Color_Coordinate_Transform/output_420_y.txt", "w");
81     fd_c_out = $fopen("C:/Users/david/Desktop/Color_Coordinate_Transform/output_420_cbcr.txt", "w");
82
83     if (fd_y_out == 0 || fd_c_out == 0) begin
84         $display("Error: Failed to create output files.");
85         $finish;
86     end
87
88     // fork-join 啟動兩個平行的監聽迴圈
89     fork
90     forever begin
91         @(posedge clk);
92         if (valid_y_out) begin
93             $fwrite(fd_y_out, "%02X\n", y_out);
94         end
95     end
96
97     forever begin
98         @(posedge clk);
99         if (valid_c_out) begin
100             $fwrite(fd_c_out, "%02X%02X\n", cb_out, cr_out);
101         end
102     end
103     join
104 end
105
106 // =====
107 // 讀取輸入檔並模擬
108 // =====
109 initial begin
110     rst_n      = 0;
111     valid_in   = 0;
112     y_in       = 0;
113     cb_in      = 0;
114     cr_in      = 0;
115
116     active_width = IMG_WIDTH;
117     active_height = IMG_HEIGHT;
118
119     // 讀取上一個模組產生的 4:4:4 YCbCr .txt檔
120     fd_in = $fopen("C:/Users/david/Desktop/Color_Coordinate_Transform/ycbcr_read.txt", "r");
121     if (fd_in == 0) begin
122         $display("Error: output_ycbcr_hex.txt not found. Please check the file path.");
123         $finish;
124     end
125
126     #(CLK_PERIOD * 2);
127     rst_n = 1;
128     #(CLK_PERIOD * 2);

```

```

130 while (!$feof(fd_in)) begin
131     @(negedge clk);
132     scan_res = $fscanf(fd_in, "%h\n", ycbcr_data);
133     if (scan_res == 1) begin
134         valid_in = 1;
135         y_in      = ycbcr_data[23:16];
136         cb_in     = ycbcr_data[15:8];
137         cr_in     = ycbcr_data[7:0];
138     end else begin
139         valid_in = 0;
140     end
141 end
142
143 @(negedge clk);
144 valid_in = 0;
145
146 #(CLK_PERIOD * 5);
147
148 $fclose(fd_in);
149 $fclose(fd_y_out);
150 $fclose(fd_c_out);
151 $display("=====");
152 $display("4:2:0 Subsampling simulation completed!");
153 $display("Please check output_420_y.txt and output_420_cbcr.txt");
154 $display("=====");
155 $finish;
156 end
157
158 endmodule

```

4. Top module (verilog)

Verilog :

```

23 module rgb2ycbcr_top #(
24     parameter MAX_WIDTH  = 1920,
25     parameter MAX_HEIGHT = 1080
26 ) (
27
28     input wire      clk,
29     input wire      rst_n,
30
31     input wire [15:0] active_width,
32     input wire [15:0] active_height,
33
34     input wire      valid_in,
35     input wire [7:0] r_in,
36     input wire [7:0] g_in,
37     input wire [7:0] b_in,
38
39     output wire      valid_y_out,
40     output wire [7:0] y_out,
41
42     output wire      valid_c_out,
43     output wire [7:0] cb_out,
44     output wire [7:0] cr_out
45 );
46
47 wire      core_valid_out;
48 wire [7:0] core_y_out;
49 wire [7:0] core_cb_out;
50 wire [7:0] core_cr_out;

```

```

52  rgb2ycbcr_core u_core (
53      .clk          (clk),
54      .rst_n        (rst_n),
55      .valid_in     (valid_in),
56      .r_in         (r_in),
57      .g_in         (g_in),
58      .b_in         (b_in),
59
60      .valid_out    (core_valid_out),
61      .y_out        (core_y_out),
62      .cb_out       (core_cb_out),
63      .cr_out       (core_cr_out)
64  );
65
66  ycbcr_444_to_420_drop #(
67      .MAX_WIDTH    (MAX_WIDTH),
68      .MAX_HEIGHT   (MAX_HEIGHT)
69  ) u_drop (
70      .clk          (clk),
71      .rst_n        (rst_n),
72      .active_width  (active_width),
73      .active_height (active_height),
74
75      .valid_in     (core_valid_out),
76      .y_in         (core_y_out),
77      .cb_in        (core_cb_out),
78      .cr_in        (core_cr_out),
79
80      .valid_y_out   (valid_y_out),
81      .y_out         (y_out),
82      .valid_c_out   (valid_c_out),
83      .cb_out        (cb_out),
84      .cr_out        (cr_out)
85  );
86  endmodule

```

Testbench :

```

23  module tb_rgb2ycbcr_top;
24
25      parameter CLK_PERIOD = 10;
26
27      // 設定圖片的真實解析度 (與讀取的.txt 對應)
28      parameter IMG_WIDTH  = 1080;
29      parameter IMG_HEIGHT = 1350;
30
31      reg      clk;
32      reg      rst_n;
33      reg [15:0] active_width;
34      reg [15:0] active_height;
35
36      reg      valid_in;
37      reg [7:0] r_in;
38      reg [7:0] g_in;
39      reg [7:0] b_in;
40
41      wire      valid_y_out;
42      wire [7:0] y_out;
43      wire      valid_c_out;
44      wire [7:0] cb_out;
45      wire [7:0] cr_out;
46
47      integer fd_in;
48      integer fd_y_out;
49      integer fd_c_out;
50      integer scan_res;
51      reg [23:0] rgb_data; // 暫存從 .txt 讀取的 24-bit RGB 資料
52

```



```

53 rgb2ycbcr_top #(
54     .MAX_WIDTH(1920),
55     .MAX_HEIGHT(1080)
56 ) u_top (
57     .clk          (clk),
58     .rst_n        (rst_n),
59     .active_width  (active_width),
60     .active_height (active_height),
61
62     .valid_in     (valid_in),
63     .r_in         (r_in),
64     .g_in         (g_in),
65     .b_in         (b_in),
66
67     .valid_y_out  (valid_y_out),
68     .y_out        (y_out),
69     .valid_c_out  (valid_c_out),
70     .cb_out       (cb_out),
71     .cr_out       (cr_out)
72 );
73
74 initial begin
75     clk = 0;
76     forever #(CLK_PERIOD / 2) clk = ~clk;
77 end
78
79 // =====
80 // 監聽結果並寫入輸出檔
81 // =====
82 initial begin
83     fd_y_out = $fopen("C:/Users/david/Desktop/Color_Coordinate_Transform/output_top_y.txt", "w");
84     fd_c_out = $fopen("C:/Users/david/Desktop/Color_Coordinate_Transform/output_top_cbcr.txt", "w");
85
86     if (fd_y_out == 0 || fd_c_out == 0) begin
87         $display("Error: Failed to create output files.");
88         $finish;
89     end
90
91 // 平行操作
92 fork
93     // 監聽 Y 結果
94     forever begin
95         @(posedge clk);
96         if (valid_y_out) begin
97             $fwrite(fd_y_out, "%02X\n", y_out);
98         end
99     end
100
101     // 監聽 CbCr 結果
102     forever begin
103         @(posedge clk);
104         if (valid_c_out) begin
105             $fwrite(fd_c_out, "%02X%02X\n", cb_out, cr_out);
106         end
107     end
108 join
109 end
110
111 // =====
112 // 讀取輸入檔 .txt 並模擬
113 // =====
114 initial begin
115     rst_n = 0;
116     valid_in = 0;
117     r_in = 0;
118     g_in = 0;
119     b_in = 0;
120
121     active_width = IMG_WIDTH;
122     active_height = IMG_HEIGHT;
123
124     // 開啟原始的 RGB 文字檔
125     fd_in = $fopen("C:/Users/david/Desktop/Color_Coordinate_Transform/road.txt", "r");
126     if (fd_in == 0) begin
127         $display("Error: road.txt not found. Please check the file path.");
128         $finish;
129     end
130
131     #(CLK_PERIOD * 2);
132     rst_n = 1;
133     #(CLK_PERIOD * 2);

```

```

135 while (!$feof(fd_in)) begin
136     @(negedge clk);
137     scan_res = $fscanf(fd_in, "%h\n", rgb_data);
138     if (scan_res == 1) begin
139         valid_in = 1;
140         r_in = rgb_data[23:16];
141         g_in = rgb_data[15:8];
142         b_in = rgb_data[7:0];
143     end else begin
144         valid_in = 0;
145     end
146 end
147
148 @(negedge clk);
149 valid_in = 0;
150
151 #(CLK_PERIOD * 10);
152
153 // 關閉檔案並結束模擬
154 $fclose(fd_in);
155 $fclose(fd_y_out);
156 $fclose(fd_c_out);
157 $display("=====");
158 $display("System-Level Top Module Simulation Completed!");
159 $display("Data Path: RGB -> YCbCr Core -> 4:2:0 Subsampler");
160 $display("Please check output_top_y.txt and output_top_cbcr.txt");
161 $display("=====");
162 $finish;
163 end
164 endmodule

```

原始 RGB 檔轉換為 4:2:0 drop 的 Y 與 CbCr 檔：

road_top_y.txt	
1457975	6e
1457976	97
1457977	91
1457978	83
1457979	7b
1457980	81
1457981	78
1457982	51
1457983	33
1457984	33
1457985	6d
1457986	60
1457987	6b
1457988	7d
1457989	7d
1457990	7b
1457991	74
1457992	60
1457993	75
1457994	73
1457995	7b
1457996	77
1457997	90
1457998	a1
1457999	8f
1458000	9c
1458001	

road_top_cbcr.txt	
364475	6274
364476	6474
364477	6274
364478	5c76
364479	5177
364480	4a78
364481	4777
364482	4b77
364483	5975
364484	5376
364485	5075
364486	4c75
364487	4b74
364488	4d74
364489	4d76
364490	5276
364491	5b77
364492	6377
364493	6577
364494	6178
364495	5878
364496	5178
364497	4e77
364498	4d76
364499	4b76
364500	4976
364501	

可以發現 CbCr 檔的總行數為 Y 檔的 1/4，即 4:2:0 的 drop 策略。

5. verify 4:2:0 and compute PSNR (python)

```
1  import numpy as np
2  import matplotlib.pyplot as plt
3  import math
4  from PIL import Image
5
6  # 確保長寬與原始 road 圖片一致
7  WIDTH = 1080
8  HEIGHT = 1350
9
10 # =====
11 # verilog 輸出結果路徑
12 # =====
13 y_file = r"C:\Users\david\Desktop\Color_Coordinate_Transform\road_top_y.txt"
14 c_file = r"C:\Users\david\Desktop\Color_Coordinate_Transform\road_top_cbcr.txt"
15
16 # 原始 .jpg 圖片路徑
17 original_img_file = r"C:\Users\david\Desktop\Color_Coordinate_Transform\road.jpg"
18
19 print("正在讀取硬體 4:2:0 輸出檔案...")
20
21 # 讀取 Y
22 try:
23     with open(y_file, 'r') as f:
24         y_data = [int(line.strip(), 16) for line in f]
25 except FileNotFoundError:
26     print(f"× 找不到 {y_file} !")
27     exit()
28
29 # 讀取 CbCr
30 try:
31     with open(c_file, 'r') as f:
32         lines = f.readlines()
33         cb_data = [int(line.strip()[0:2], 16) for line in lines]
34         cr_data = [int(line.strip()[2:4], 16) for line in lines]
35 except FileNotFoundError:
36     print(f"× 找不到 {c_file} !")
37     exit()
38
39 # 防呆
40 expected_y_len = WIDTH * HEIGHT
41 expected_c_len = (WIDTH // 2) * (HEIGHT // 2)
42
43 if len(y_data) != expected_y_len or len(cb_data) != expected_c_len:
44     print("× 錯誤：檔案內的資料行數與設定的解析度不符！")
45     print(f"預期 Y: {expected_y_len} 行，實際: {len(y_data)} 行")
46     print(f"預期 C: {expected_c_len} 行，實際: {len(cb_data)} 行")
47     exit()
48
49 # =====
50 # 重組 2D YCbCr
51 # =====
52 print("正在進行色度上採樣與陣列重組...")
53
54 # 將一維資料折疊成二維矩陣
55 y_img = np.array(y_data, dtype=np.float32).reshape((HEIGHT, WIDTH))
56 cb_img = np.array(cb_data, dtype=np.float32).reshape((HEIGHT // 2, WIDTH // 2))
57 cr_img = np.array(cr_data, dtype=np.float32).reshape((HEIGHT // 2, WIDTH // 2))
58
59 # CbCr共用
60 cb_upsampled = np.repeat(np.repeat(cb_img, 2, axis=0), 2, axis=1)
61 cr_upsampled = np.repeat(np.repeat(cr_img, 2, axis=0), 2, axis=1)
62
```

```

63 # =====
64 # YCbCr 還原 RGB
65 # =====
66 print("正在執行 YCbCr 到 RGB 的逆向轉換...")
67
68 Cb = cb_upsampled - 128.0
69 Cr = cr_upsampled - 128.0
70
71 R = y_img + 1.402 * Cr
72 G = y_img - 0.344136 * Cb - 0.714136 * Cr
73 B = y_img + 1.772 * Cb
74
75 # =====
76 # 截斷
77 # =====
78 rgb_array = np.stack((R, G, B), axis=2)
79 rgb_array = np.clip(rgb_array, 0, 255).astype(np.uint8)
80
81 # =====
82 # 計算 PSNR
83 # =====
84 print("正在計算 PSNR 數據...")
85
86 try:
87     # 讀取原始圖片並轉成 numpy 陣列
88     original_img = Image.open(original_img_file).convert('RGB')
89
90     # 防呆
91     if original_img.size != (WIDTH, HEIGHT):
92         print(f"⚠ 警告：原圖尺寸 {original_img.size} 與設定的 ({WIDTH}, {HEIGHT}) 不符！")
93
94     original_array = np.array(original_img, dtype=np.float32)
95     hw_array = rgb_array.astype(np.float32) # 轉成 float32 避免相減時溢位
96
97     # 計算 MSE
98     mse = np.mean((original_array - hw_array) ** 2)
99
100     if mse == 0:
101         print("PSNR: 無限大 (與原圖完全 100% 一致)")
102     else:
103         max_pixel = 255.0
104         psnr = 10 * math.log10((max_pixel ** 2) / mse)
105         print(f"-----")
106         print(f"MSE (均方誤差): {mse:.4f}")
107         print(f"PSNR (峰值信噪比): {psnr:.2f} dB")
108         print(f"-----")
109 except FileNotFoundError:
110     print(f"✗ 找不到原始圖片 {original_img_file}，無法計算 PSNR。")
111
112 print("轉換完成！正在顯示圖片...")
113 plt.figure(figsize=(10, 8))
114 plt.imshow(rgb_array)
115 plt.title(f"Reconstructed RGB (4:2:0 Drop Strategy)\nPSNR: {psnr:.2f} dB" if 'psnr' in locals() else "Reconstructed RGB")
116 plt.axis('off')
117 plt.tight_layout()
118 plt.show()

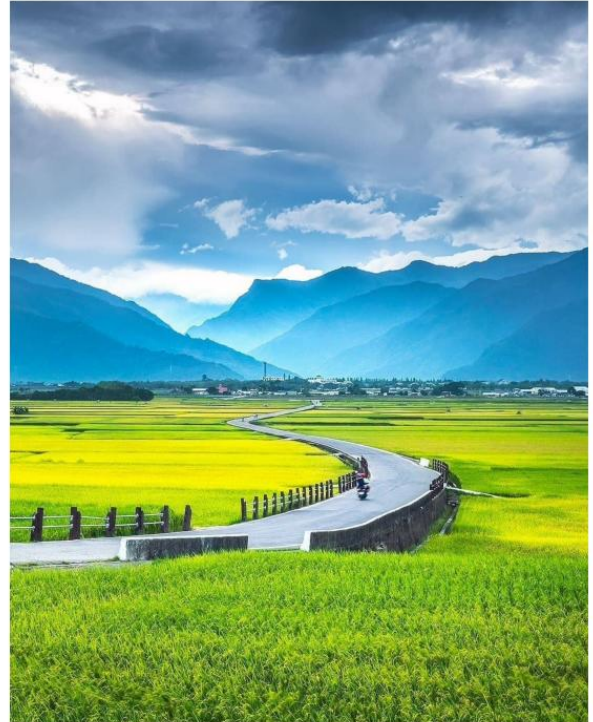
```

原始圖片：



4:2:0 drop YCbCr：

Reconstructed RGB (4:2:0 Drop Strategy)
PSNR: 40.26 dB



可以發現即使丟掉了 75% 的色度資訊，人眼其實還是分辨不出兩者的差異！

計算 MSE 與 PSNR 的結果：(PSNR>40dB 代表幾乎無損品質)

```
正在進行色度上採樣與陣列重組...
正在執行 YCbCr 到 RGB 的逆向轉換...
正在計算 PSNR 數據...

-----
MSE (均方誤差): 6.1298
PSNR (峰值信噪比): 40.26 dB
-----

轉換完成！正在顯示圖片...
PS C:\Users\david\Desktop\Color_Coordinate_Transform> |
```

Discussion

這次的 Color Coordinate Transform 專案，讓我了解軟體思維與硬體思維的巨大差異。在軟體中，呼叫浮點數運算和處理陣列是理所當然的，但在數位電路設計裡，每一個小數點、每一次除法，都代表著昂貴的邏輯閘與面積成本。

透過這個專案，我實作了運用定點數量化 fixed-point 與 pipeline 的技術，用極低的硬體代價去逼近複雜的數學公式，其中最讓我印象深刻的是實作 4:2:0 降採樣的過程，僅僅是透過對空間座標最低位元 LSB 的精準判斷與控制，就能在維持視覺無損品質 (>40 dB PSNR) 的前提下，成功省下 75% 的色度記憶體頻寬。

最後，實際使用 python 與 verilog 的軟硬體協同設計，並看到硬體產出的像素陣列與軟體理論值高度吻合，這不但讓我非常開心這次的專案有好的成果，也體會到高等數位訊號處理課程中影像處理/壓縮的有趣之處。